

# **UNIVERSIDAD PERUANA LOS ANDES**

**FACULTAD DE INGENIERÍA**

**Escuela Profesional de Ingeniería de Sistemas y  
Computación**

**Asignatura: Algoritmos y Estructuras de Datos**

**Mg. Jaime Ortiz Fernández**

e-mail: [d.jortiz@ms.upla.edu.pe](mailto:d.jortiz@ms.upla.edu.pe)

**HUANCAYO - PERÚ**

A teal-colored decorative curve is located in the bottom right corner of the slide, starting from the bottom edge and curving upwards and to the left.

## UNIDAD III

## CONCEPTOS DE ALGORITMOS Y ESTRUCTURAS DE DATOS

### ***TEMA: Clase ArrayList***

Diapositivas: Mg. Jaime Ortíz Fernández

A teal-colored decorative curve is located in the bottom right corner of the slide, starting from the bottom edge and curving upwards and to the left.

# Paquetes

- *Paquetes*: organizan las clases en una jerarquía de paquetes y subpaquetes
- Para indicar que una clase pertenece a un paquete o subpaquete se utiliza la palabra *package* al principio de la clase

```
package paquete1.subpaquete1;  
class MiClase {
```

- Para utilizar clases de un paquete en otro, se colocan al principio sentencias *import* con los paquetes necesarios:

```
package otra paquete;  
import paquete1.subpaquete1.MiClase;  
import java.util.*;  
class MiOtraClase {
```

# Paquetes

- Si no utilizamos sentencias *import*, deberemos escribir el nombre completo de cada clase del paquete no importado (incluyendo subpaquetes)

```
class MiOtraClase {  
    paquete1.subpaquete1.MiClase a = ...;    // Sin import  
    MiClase a = ...;                          // Con import
```

- Los paquetes se estructuran en directorios en el disco duro, siguiendo la misma jerarquía de paquetes y subpaquetes

```
./paquete1/subpaquete1/MiClase.java
```

# Introducción

- Java proporciona un amplio conjunto de clases útiles para desarrollar aplicaciones
  - En esta sesión veremos algunos grupos de ellas:
    - Clases útiles para crear y gestionar diferentes tipos de colecciones
    - Clases para recorrer, ordenar y manipular las colecciones
    - Clases para encapsular nuestros tipos de datos
- 
- A teal-colored decorative shape, resembling a quarter-circle or a stylized wave, is located in the bottom right corner of the slide.

# Colecciones

En el paquete `java.útil` se tienen clases con colecciones de objetos

Representan grupos de objetos, llamados elementos

Podemos encontrar de distintos tipos, según si sus elementos están ordenados, si permiten repetir elementos, etc.

La interfaz `Collection` define el esqueleto que deben tener todos los tipos de colecciones

Por tanto, todos tendrán métodos generales como:

- `boolean add(Object o)`
- `boolean remove(Object o)`
- `boolean contains(Object o)`
- `void clear()`
- `boolean isEmpty()`
- `Iterator iterator()`
- `int size()`
- `Object[] toArray()`

# Listas

- Cuando se escribe un programa que colecciona datos, no siempre se sabe cuántos valores se tendrá.
- En tal caso una lista ofrece dos ventajas significativas:
  - La lista puede crecer o disminuir como sea necesario.
  - La clase `ArrayList` ofrece métodos para las operaciones comunes, tal como insertar o eliminar elementos.
- Las listas con una *clase genérica* (puede contener muchos tipos de objetos) que se encuentra en el paquete `java.util.ArrayList`

# Listas de elementos

- La interfaz `List` hereda de `Collection`
  - Operaciones propias de una colección tipo *lista*
  - Los elementos tienen un orden (posición en la lista)
- Así, tendremos otros nuevos métodos, además de los de `Collection`:
  - `void add(int posicion, Object o)`
  - `Object get(int indice)`
  - `int indexOf(Object o)`
  - `Object remove(int indice)`
  - `Object set(int indice, Object o)`

# Tipos de listas

- ArrayList: implementa una lista de elementos mediante un *array* de tamaño variable
  - *NO sincronizado*
- Vector: existe desde las primeras versiones de Java, después se acomodó al marco de colecciones implementando la interfaz List.
  - *Similar a ArrayList, pero SINCRONIZADO. Tiene métodos anteriores a la interfaz List:*
    - `void addElement(Object o) / boolean removeElement(Object o)`
    - `void insertElementAt(Object o, int posicion)`
    - `void removeElementAt(Object o, int posicion)`
    - `Object elementAt(int posicion)`
    - `void setElementAt(Object o, int posicion)`
    - `int size()`
- LinkedList: lista doblemente enlazada. Útil para simular pilas o colas
  - `void addFirst(Object o) / void addLast(Object o)`
  - `Object getFirst() / Object getLast()`
  - `Object removeFirst() / Object removeLast()`

# Uso de ArrayList

- Durante la declaración se indica el tipo de los elementos.

- Dentro de < > como el tipo de “parámetro”

- El tipo debe ser una clase

- No se puede usar tipos de datos primitivos (int, double...)

```
ArrayList<Objeto> variable = new ArrayList<Objeto>();
```

```
ArrayList<String> names = new ArrayList<String>();
```

- Métodos útiles de ArrayList

- add: añade un elemento

- get: retorna un elemento

- remove: elimina un elemento

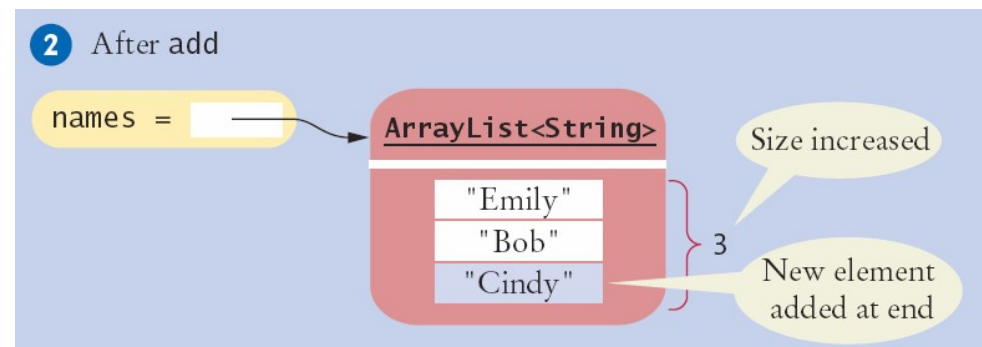
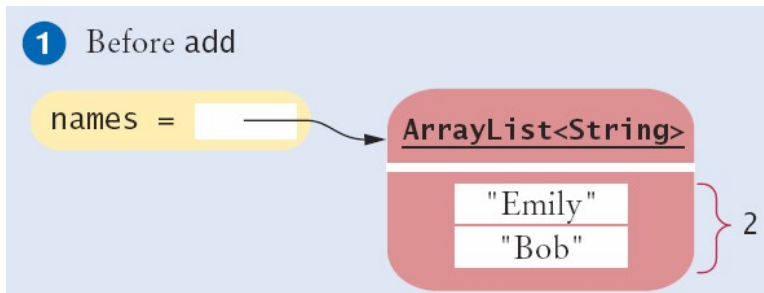
- set: cambia un elemento

- size: longitud del ArrayList

# Inserción de un elemento con add()

- El método `add` tiene dos versiones (es sobrecargado):
- **Primera versión o forma:**
  - Pasar un elemento para añadirse al final

```
names.add("Cindy");
```

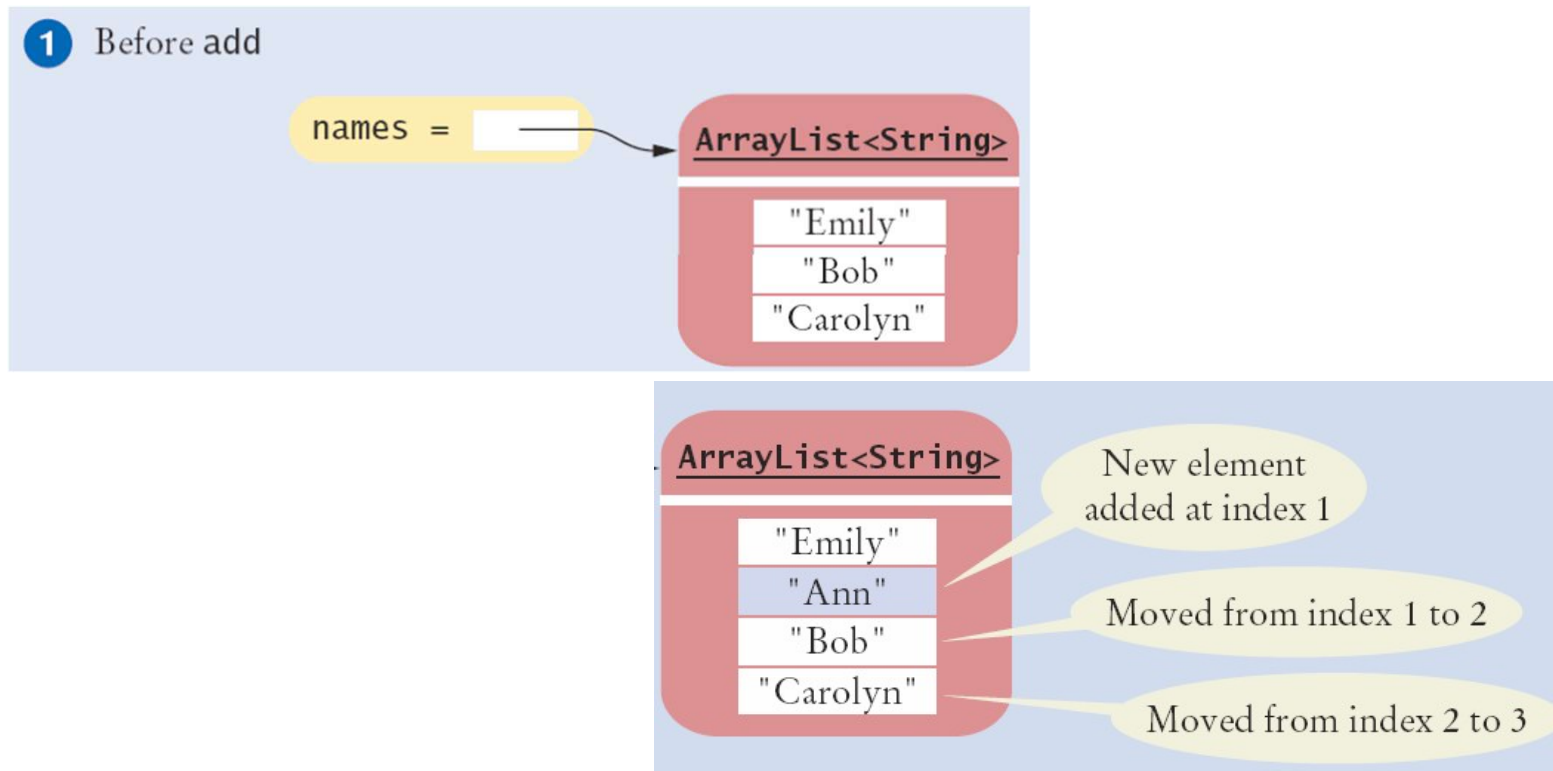


# Inserción de un elemento con add()

## Segunda versión o forma:

- Pasar un índice y el nuevo elemento a añadir en esa posición. Los otros elementos se mueven hacia adelante.

```
names.add(1, "Ann");
```



# Uso de ArrayList

```
ArrayList<String> nombres= new ArrayList<String>();

nombres.add("Ana"); // agrega Ana al final de la lista
nombres.add("Cindy"); // agrega Cindy al final de la lista

System.out.println(nombres); // muestra los datos

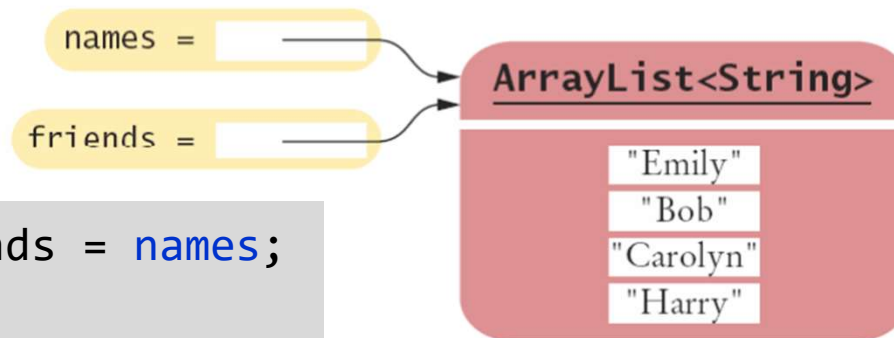
nombres.add(1,"Bob"); //agrega Bob en la posición 1
nombres.remove(0); // elimina el objeto de la pos 0
nombres.set(0, "Bill"); // cambia el objeto de la pos 0
con Bill
String name = nombres.get(i); // asigna a name el objeto
de la pos 0

String last = nombres.get(nombres.size() - 1); // recupera el ultimo objeto

String nombres = nombres.get(i);
String last     = nombres.get(nombres.size() - 1);
```

## Copia de ArrayList

- ArrayList mantiene una referencia como los arrays.
- Copiando una referencia:



```
ArrayList<String> friends = names;  
friends.add("Harry");
```

- Para hacer una copia, pasar la referencia del ArrayList original al constructor del nuevo:

referencia

```
ArrayList<String> newNames = new ArrayList<String>(names);
```

## ArrayList y métodos

- De igual manera que los arrays, un ArrayList puede ser usado como parámetro o valor retornado.
- Ejemplo: método que recibe un ArrayList y devuelve la lista invertida



referencia

```
public static ArrayList<String> reverse(ArrayList<String> names)
{
    // Crea una lista para el resultado del metodo
    ArrayList<String> result = new ArrayList<String>();
    // Recorre la lista de nombres en orden inverso (último a primero)
    for (int i = names.size() - 1; i >= 0; i--)
    {
        // Añade cada nombre al resultado
        result.add(names.get(i));
    }
    return result;
}
```

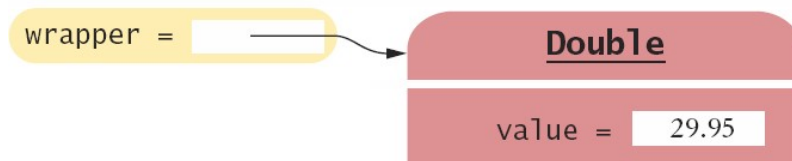
---

# Wrappers y auto-boxing

Java ofrece la clase wrapper para tipos primitivos.

- Las conversiones son automáticas usando *auto-boxing*
- El tipo primitivo se convierte a la clase Wrapper

```
double x = 29.95;  
Double wrapper;  
wrapper = x; // auto-boxing
```



- Clase Wrapper a tipo primitivo

```
double x;  
Double wrapper = 29.95;  
x = wrapper; // unboxing
```

| Primitive Type | Wrapper Class |
|----------------|---------------|
| byte           | Byte          |
| boolean        | Boolean       |
| char           | Character     |
| double         | Double        |
| float          | Float         |
| int            | Integer       |
| long           | Long          |
| short          | Short         |

# Wrappers y auto-boxing

- No se puede usar tipos primitivos en un ArrayList, pero se puede usar sus clases wrapper (envolventes).
  - Depende del auto-boxing para la conversión
- Declarar el ArrayList con clases wrapper para los tipos primitivos

```
double x = 19.95;  
ArrayList<Double> data = new ArrayList<Double>();  
data.add(19.95);           // boxing  
data.add(x);               // boxing  
double x = data.get(0);    // unboxing
```

# Algoritmos con ArrayList

- La conversión de arrays a ArrayList requiere el cambio de:

- uso de índices `[i]`
- `data.length`

```
double largest = data[0];
for (int i = 1; i < data.length; i++)
{
    if (data[i] > largest) {
        largest = data[i];
    }
}
```

- a
- métodos `get()`
- `data.size()`

```
double largest = data.get(0);
for (int i = 1; i < data.size(); i++)
{
    if (data.get(i) > largest) {
        largest = data.get(i);
    }
}
```

# Cuándo usar Arrays o ArrayList

- Usar arrays si:
  - el tamaño del array nunca cambia
  - se tiene una lista grande de tipos primitivos
  - lo pide el “jefe”, nos ordenan usar esa estructura
- Usar un ArrayList
  - en cualquiera de los otros casos
  - especialmente si se tiene un número desconocido de valores de entrada

## Cuidado con length o size

No hay consistencia para determinar el número de elementos en un Array, ArrayList o String

| Data Type  | Number of Elements      |
|------------|-------------------------|
| Array      | <code>a.length</code>   |
| Array list | <code>a.size()</code>   |
| String     | <code>a.length()</code> |



Gracias por su Atención



Preguntas?